AI Application Architecture

Week 1, Assignment 5 — AI Workspace Application

Hooria Zahoor | BS Artificial Intelligence, The University of Faisalabad

1. Architecture Diagram

The AI Workspace application (built in Assignment 3) follows a straightforward, layered request/response architecture. Each layer has a single, well-defined responsibility, which keeps the system easy to debug and extend:

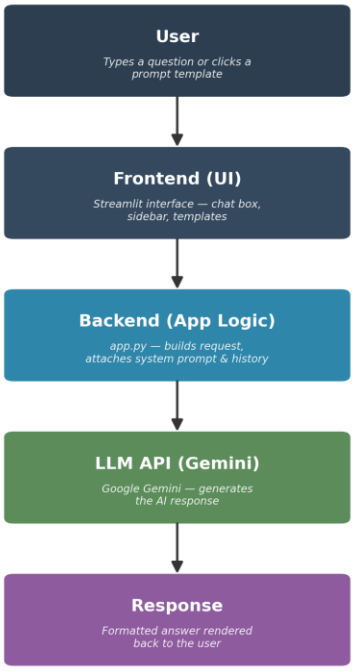

Figure 1: Layered architecture of the AI Workspace application.

2. Request Flow

A request begins when the user types a message (or selects a prompt template) in the Streamlit frontend and clicks Send. The frontend passes this raw text to the backend logic in app.py. The backend then:

- Validates the input — rejects the request early if the message is empty.
- Retrieves the active system prompt (preset or custom) and the full conversation history for the current chat session.
- Assembles a single messages array in the format the API expects: [{role: 'system', ...}, {role: 'user', ...}, {role: 'assistant', ...}, ...].
- Sends this payload to the Gemini API (via its OpenAI-compatible endpoint) along with the selected model name and temperature setting, using the OpenAI Python client.

3. Response Flow

Once the LLM API processes the request, it returns a response object containing the generated text and token-usage metadata. The backend then:

- Extracts the response text from response.choices[0].message.content.
- Reads token usage from response.usage.total_tokens and adds it to a running session counter.
- Records the response time (measured from just before the API call to just after it returns).
- Appends the assistant's message — along with its metadata (time, tokens, model used) — to the session's conversation history.
- Triggers a UI refresh so the frontend re-renders the chat, displaying the new message with Markdown formatting (bold text, code blocks, lists, etc. all render correctly).

4. Error Handling

Because network calls and external APIs can fail in several distinct ways, the backend wraps every API call in a try/except block with specific handlers rather than a single generic catch-all:

- Empty prompt — caught before any API call is made; the user sees a warning and no request is sent, avoiding wasted API calls.
- Invalid or missing API key — caught via AuthenticationError; shown as a clear, actionable message asking the user to check their key in the sidebar.
- Connection failure — caught via APIConnectionError, for cases like no internet access or the API endpoint being unreachable.
- Rate limits — caught via RateLimitError, informing the user they've hit a usage quota and should wait.
- Any other failure — caught by a final generic Exception handler, so the app never crashes outright; it always degrades to a readable error message inside the chat instead.

This layered error handling means a failure at any point in the pipeline is shown to the user in plain language, rather than as a raw stack trace or a frozen interface.

5. API Integration

The application integrates with Google's Gemini models through Gemini's OpenAI-compatible endpoint (https://generativelanguage.googleapis.com/v1beta/openai/). This is a deliberate architectural choice: by pointing the standard openai Python client at this base URL and supplying a Gemini API key instead of an OpenAI key, the application gains access to a free-tier LLM without needing any Gemini-specific SDK or code path. The model name (e.g., gemini-2.5-flash) and generation parameters (temperature) are configurable from the sidebar, so switching models does not require any code changes — only a different value passed into the same client.chat.completions.create() call.